

Windows Patch Management Guide

Production-Grade Enterprise Guide

Author: Krishna Chaithanya Yada

LinkedIn: [linkedin.com/in/krishnachaithanyayada](https://www.linkedin.com/in/krishnachaithanyayada)

GitHub: github.com/yadakrishna245

Version 3.0 | June 2026

© 2026 Krishna Chaithanya Yada. All Rights Reserved.

[image] [image] [image] [image] [image]

Windows Patch Management Guide — Production Grade

Enterprise-grade patch management procedures for Windows Server environments. Covers SCCM/MECM, WSUS, Intune, Ansible, and PDQ Deploy with exact paths, commands, and step-by-step instructions.

Table of Contents

#	Section	Description
1	Overview	What is patching, why it matters
2	Tools Comparison	SCCM vs WSUS vs Intune vs Ansible vs PDQ
3	SCCM/MECM	Primary enterprise method
4	WSUS	Standalone/small environments
5	Microsoft Intune	Cloud-based patching
6	Ansible	Automation at scale
7	PDQ Deploy	SMB alternative
8	Pre-Patching Checklist	Health checks before patching
9	Post-Patching Validation	Verification steps
10	Rollback Procedures	Undo failed patches
11	Troubleshooting	30+ common scenarios
12	Automation Scripts	Ready-to-use scripts
13	Best Practices	Do's and Don'ts
14	Change Management	ITIL-aligned process
15	Quick Reference	Command cheat sheet

Section 1: Overview

What is Windows Patching?

Windows patching is the process of applying software updates released by Microsoft to fix security vulnerabilities, improve stability, and add features to Windows operating systems and applications.

Why Patching Matters

Reason	Impact
Security	Prevents exploitation of known vulnerabilities (CVEs)
Compliance	Required for PCI-DSS, HIPAA, SOX, ISO 27001, NIST
Stability	Fixes bugs, improves performance
Supportability	Microsoft only supports patched systems
Insurance	Many cyber-insurance policies require timely patching

Types of Windows Patches

Patch Type	Description	Frequency
Security Updates	Fix specific CVEs	Monthly (Patch Tuesday)
Critical Updates	Non-security critical fixes	As needed
Cumulative Updates (CU)	Bundle of all monthly fixes	Monthly
Feature Updates	Major OS upgrades (e.g., 21H2 → 22H2)	Semi-annual
Driver Updates	Hardware driver improvements	As needed
.NET Updates	Framework security & bug fixes	Monthly
Servicing Stack Updates (SSU)	Updates the update mechanism itself	As needed

Patch Tuesday Explained

- **When:** Second Tuesday of every month
- **Time:** Released at 10:00 AM PST (17:00 UTC)
- **What:** Security and quality updates for all supported Windows versions
- **Out-of-Band:** Emergency patches released outside schedule for critical zero-days

Windows Server Versions Covered

Version	Build	End of Support	Status
Windows Server 2012 R2	6.3.9600	Oct 2023 (ESU available)	Extended Security Updates
Windows Server 2016	14393	Jan 2027	Mainstream
Windows Server 2019	17763	Jan 2029	Mainstream
Windows Server 2022	20348	Oct 2031	Mainstream
Windows Server 2025	26100	Oct 2034	Mainstream

Section 2: Patching Tools Comparison

Feature Comparison Matrix

Feature	SCCM/MECM	WSUS	Intune	Ansible	PDQ Deploy
Scale	500-100,000+	1-5,000	1-50,000+	1-10,000+	1-500
Cost	Enterprise License	Free (built-in)	Per-user license	Free (open source)	\$500-1,500/yr
Cloud Support	Hybrid	On-prem only	Cloud-native	Both	On-prem only
Reporting	Excellent	Basic	Good	Custom	Good
Complexity	High	Medium	Low	Medium-High	Low
Agent Required	Yes (SCCM Client)	No (built-in WUA)	Yes (Intune agent)	No (agentless/WinRM)	Yes (PDQ agent)
Linux Support	Limited	No	No	Yes	No
Approval Workflow	Yes	Yes	Yes	Custom	Yes

When to Use Which Tool

Environment Size	Recommended Tool	Reason
Small (1-50 servers)	WSUS or PDQ Deploy	Free, simple setup
Medium (50-500 servers)	SCCM/MECM or Ansible	Scalable, good reporting
Large (500-5,000+ servers)	SCCM/MECM + Ansible	Enterprise features, automation
Cloud/Hybrid	Microsoft Intune	Azure AD integration, no VPN needed
Multi-OS (Windows + Linux)	Ansible	Single tool for all platforms

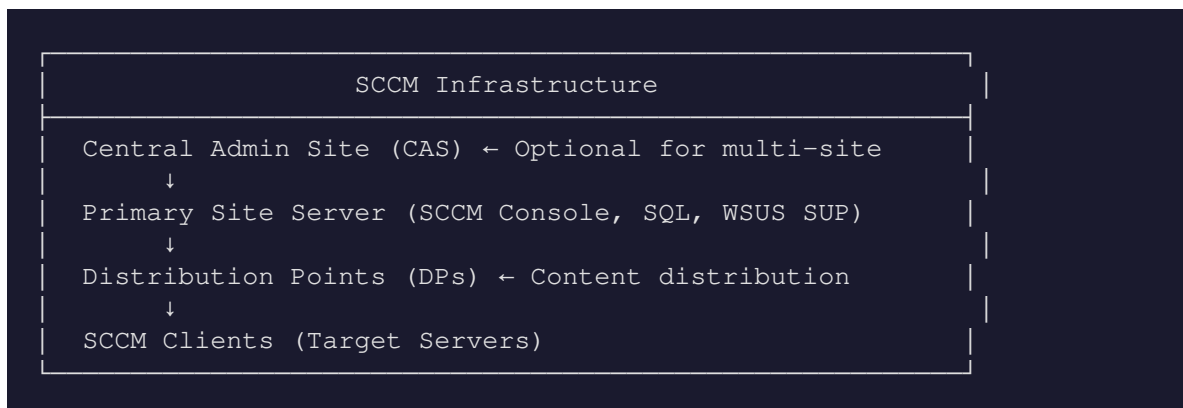
Pros and Cons Summary

Tool	Pros	Cons
SCCM/MECM	Full lifecycle management, compliance reporting, maintenance windows	Expensive, complex setup, requires SQL Server
WSUS	Free, built into Windows Server, GPO integration	Limited reporting, no maintenance windows, manual approval

Intune	Cloud-native, no infrastructure, automatic updates	Requires Azure AD, limited server support, per-user cost
Ansible	Agentless, cross-platform, Infrastructure as Code	Requires Linux control node, learning curve, custom reporting
PDQ Deploy	Easy GUI, fast deployment, good for SMB	Windows only, limited scale, annual license

Section 3: SCCM/MECM (Primary Method)

Architecture Overview



Key Paths on SCCM Server: - SCCM Install: C:\Program Files\Microsoft Configuration Manager - **SCCM Logs:** C:\Program Files\Microsoft Configuration Manager\Logs - **Content Library:** E:\SCCMContentLib - **Update Packages:** \\scm-server\Sources\Updates\YYYY-MM - **WSUS Content:** D:\WSUS_Content

Prerequisites

- Windows Server 2019/2022 for site server
- SQL Server 2019+ (Standard or Enterprise)
- WSUS role installed (Software Update Point)
- Minimum 100 GB free on content drive
- .NET Framework 4.8+
- Windows ADK installed
- Active Directory schema extended

Step-by-Step Patch Deployment

Step 1: Synchronize Software Updates

[Run on: SCCM Primary Site Server]

1. Open SCCM Console
2. Navigate: Software Library → Software Updates → All Software Updates
3. Right-click "All Software Updates" → Synchronize Software Updates
4. Monitor: Monitoring → Software Update Point Synchronization Status

Log to monitor: C:\Program Files\Microsoft Configuration Manager\Logs\wsyncmgr.log

Step 2: Create Software Update Group

1. Software Library → Software Updates → All Software Updates
2. Filter updates: Released Date = Current Month, Required > 0
3. Select relevant updates → Right-click → Create Software Update Group
4. Name: "YYYY-MM Security Updates – Windows Server"
5. Click OK

Step 3: Create Deployment Package

1. Software Library → Software Updates → Deployment Packages
2. Right-click → Create Deployment Package
3. Name: "Updates-YYYY-MM"
4. Package Source Path: \\sccm-server\Sources\Updates\YYYY-MM
5. Select Distribution Points → Add → Distribution Point Group
6. Complete wizard

Package Source Structure:

```
\\sccm-server\Sources\Updates\
├── 2026-06\
│   ├── Windows Server 2019\
│   ├── Windows Server 2022\
│   └── Windows Server 2025\
├── 2026-05\
└── 2026-04\
```

Step 4: Deploy to Collection

1. Software Library → Software Updates → Software Update Groups
2. Right-click your update group → Deploy
3. Collection: "All Windows Servers – Production"
4. Deployment Settings:
 - Type: Required
 - Send wake-up packets: Yes
5. Scheduling:
 - Available time: As soon as possible
 - Installation deadline: Saturday 02:00 AM
6. User Experience:
 - Suppress restart outside maintenance window: Yes
7. Complete wizard

Step 5: Configure Maintenance Windows

1. Assets and Compliance → Device Collections
2. Right-click target collection → Properties → Maintenance Windows
3. Add new window:
 - Name: "Monthly Patch Window – Saturday"
 - Schedule: 3rd Saturday, 02:00 – 06:00
 - Type: Software Updates

Step 6: Monitor Compliance

1. Monitoring → Deployments → Select your deployment
2. View: Compliance Statistics
3. Check: Success %, In Progress, Error, Unknown

Client-Side Validation Commands

[Run on: Target Windows Server]

```
# Check SCCM client health
Get-Service -Name CcmExec | Select-Object Status, StartType

# Force machine policy refresh
Invoke-WMIMethod -Namespace root\ccm -Class SMS_Client -Name TriggerSchedule -A

# Force software update scan
Invoke-WMIMethod -Namespace root\ccm -Class SMS_Client -Name TriggerSchedule -A

# Force software update deployment evaluation
Invoke-WMIMethod -Namespace root\ccm -Class SMS_Client -Name TriggerSchedule -A

# Check pending updates
Get-WmiObject -Namespace root\ccm\ClientSDK -Class CCM_SoftwareUpdate | Select-
```

Troubleshooting SCCM Patching

Issue	Log File	Resolution
Updates not syncing	wsyncmgr.log	Verify SUP proxy settings, restart SMS_EXECUTIVE
Content not downloading	ContentTransferManager.log	Check DP connectivity, verify package source
Client not receiving policy	PolicyAgent.log	Run machine policy cycle, check boundary groups
Updates stuck installing	UpdatesDeployment.log	Check C:\Windows\CCM\Logs\UpdatesHandler.log on client
Compliance not reporting	StateMessage.log	Force state message upload, check MP connectivity

Section 4: WSUS (Windows Server Update Services)

When to Use WSUS

- Standalone environments without SCCM
- Small to medium environments (1-5,000 servers)
- Budget-constrained (WSUS is free with Windows Server)

- Simple approval-based patching needs

Installation Step-by-Step

[Run on: WSUS Server]

Step 1: Install WSUS Role

```
# Install WSUS with Windows Internal Database (WID)
Install-WindowsFeature -Name UpdateServices -IncludeManagementTools

# OR with SQL Server backend (recommended for 500+ clients)
Install-WindowsFeature -Name UpdateServices-Services, UpdateServices-DB -IncludeManagementTools
```

Step 2: Post-Installation Configuration

```
# Run post-install to configure content directory
& "C:\Program Files\Update Services\Tools\wsusutil.exe" postinstall CONTENT_DIRECTORY
```

Key WSUS Paths: - Content Storage: D:\WSUS_Content - WSUS Database (WID): C:\Windows\WID\Data\SUSDB.mdf - WSUS Application: C:\Program Files\Update Services - WSUS Logs: C:\Program Files\Update Services\LogFiles - IIS Port: 8530 (HTTP) / 8531 (HTTPS)

Step 3: Configure WSUS Console

1. Open: Server Manager → Tools → Windows Server Update Services
2. Run Configuration Wizard:
 - Upstream Server: Microsoft Update (or upstream WSUS)
 - Products: Select Windows Server 2016, 2019, 2022, 2025
 - Classifications: Critical, Security, Update Rollups, Service Packs
 - Sync Schedule: Daily at 03:00 AM
3. First Synchronization: Approve → Start

Step 4: Approve Updates

1. WSUS Console → Updates → All Updates
2. Filter: Approval = Unapproved, Status = Needed
3. Select updates → Right-click → Approve
4. Select computer group → Approved for Install
5. Click OK

GPO Configuration for WSUS Clients

GPO Path: Computer Configuration → Policies → Administrative Templates → Windows Components → Windows Update

Policy	Setting	Value
--------	---------	-------

Configure Automatic Updates	Enabled	4 - Auto download and schedule install
Specify intranet Microsoft update service	Enabled	http://wsus-server:8530
Specify intranet statistics server	Enabled	http://wsus-server:8530
Automatic Updates detection frequency	Enabled	4 hours
No auto-restart with logged on users	Enabled	—
Allow non-administrators to receive notifications	Enabled	—
Reschedule Automatic Updates scheduled installations	Enabled	15 minutes

Create GPO: [Run on: Domain Controller]

```
# Create and link GPO
New-GPO -Name "WSUS-Client-Configuration" | New-GPLink -Target "OU=Servers,DC=d
```

Client-Side Commands

[Run on: Target Windows Server]

```
# Force detection (Legacy - Server 2012/2016)
wuaclt /detectnow
wuaclt /reportnow

# Force detection (Server 2019/2022/2025)
usoclient StartScan
usoclient StartDownload
usoclient StartInstall

# Check WSUS server registration
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" | Se

# Restart Windows Update service
Restart-Service wuauserv -Force
```

WSUS Reporting

[Run on: WSUS Server]

```
# Get update status summary
Get-WsusUpdate -Approval Approved -Status Needed | Format-Table Title, UpdatesN

# Get computer compliance
Get-WsusComputer | Select-Object FullDomainName, LastReportedStatusTime, LastSy
```

When to Use Intune

- Azure AD joined or hybrid-joined devices
- Remote/distributed workforce
- Cloud-first strategy
- No on-premises infrastructure available
- Windows 10/11 endpoints and Windows Server 2019+

Setup Steps

Step 1: Navigate to Update Rings

1. Open: <https://intune.microsoft.com>
2. Navigate: Devices → Manage updates → Windows 10 and later updates
3. Click: Update rings tab → Create profile

Step 2: Create Update Ring

Configuration:

- Name: "Production Servers - Monthly Updates"
- Description: "Monthly security updates with 7-day deferral"

Update Ring Settings:

- Microsoft product updates: Allow
- Windows drivers: Allow
- Quality update deferral period: 7 days
- Feature update deferral period: 30 days
- Upgrade Windows 10 to Latest Windows 11: No
- Set feature update uninstall period: 10 days

User Experience Settings:

- Automatic update behavior: Auto install at maintenance time
- Active hours start: 06:00
- Active hours end: 22:00
- Restart checks: Allow
- Deadline for quality updates: 3 days
- Deadline for feature updates: 7 days
- Grace period: 2 days
- Auto reboot before deadline: Yes

Step 3: Assign to Groups

1. Assignments tab:
 - Include: "SG-Windows-Servers-Production"
 - Exclude: "SG-Windows-Servers-Critical" (patch separately)
2. Click: Create

Compliance Policies

1. Devices → Compliance policies → Create policy
2. Platform: Windows 10 and later
3. Settings:
 - Require: Device to be at or under maximum OS version
 - Minimum OS version: 10.0.17763.5000 (example)

4. Actions for noncompliance:
 - Mark device noncompliant: Immediately
 - Send email notification: After 3 days
 - Retire device: After 30 days (optional)

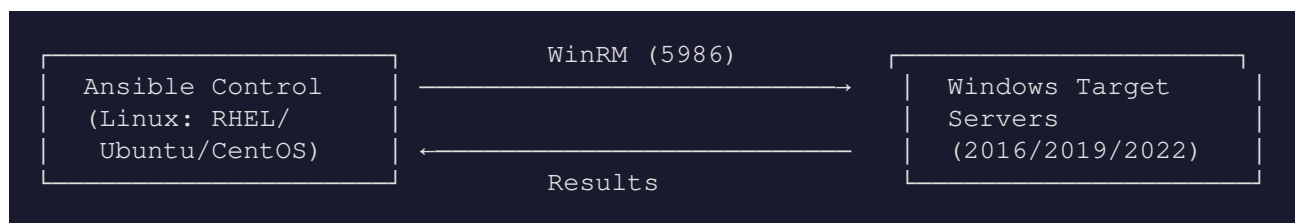
Intune Reporting

1. Reports → Windows updates → Update rings tab
2. View: Devices with errors, Devices with updates paused
3. Export: CSV for change management records

Section 6: Ansible for Windows Patching

CRITICAL: Where to install Ansible

Ansible MUST be installed on a Linux control node. It CANNOT run natively on Windows. The control node connects to Windows targets via WinRM (port 5986/HTTPS).



Control Node Setup

Install on RHEL/CentOS 8+

[Run on: Ansible Control Node (Linux)]

```
# Install EPEL repository
sudo dnf install epel-release -y

# Install Ansible
sudo dnf install ansible-core -y

# Install Windows modules collection
ansible-galaxy collection install ansible.windows
ansible-galaxy collection install community.windows

# Verify
ansible --version
```

Install on Ubuntu 22.04/24.04

[Run on: Ansible Control Node (Linux)]

```
# Add Ansible PPA
sudo apt update
sudo apt install software-properties-common -y
```

```

sudo add-apt-repository --yes --update ppa:ansible/ansible

# Install Ansible
sudo apt install ansible -y

# Install Windows modules
ansible-galaxy collection install ansible.windows
ansible-galaxy collection install community.windows

# Install pywinrm
pip3 install pywinrm

```

Directory Structure

```

/etc/ansible/
├── ansible.cfg                                # Global configuration
/opt/ansible/
├── inventory/
│   ├── production/
│   │   ├── hosts.yml                        # Production inventory
│   │   └── group_vars/
│   │       └── windows.yml                # Windows connection vars
│   └── staging/
│       ├── hosts.yml
│       ├── group_vars/
│       └── windows.yml
├── playbooks/
│   ├── patch_all_updates.yml
│   ├── patch_security_only.yml
│   ├── patch_specific_kb.yml
│   ├── pre_patch_check.yml
│   └── post_patch_validate.yml
└── roles/
    └── windows_patching/
        ├── tasks/
        ├── handlers/
        └── vars/

```

WinRM Setup on Windows Targets

[Run on: Target Windows Server]

```

# Enable WinRM with HTTPS (RECOMMENDED for production)
winrm quickconfig -transport:https -force

# OR run the Ansible-provided setup script
$url = "https://raw.githubusercontent.com/ansible/ansible/devel/examples/script
$file = "$env:temp\ConfigureRemotingForAnsible.ps1"
(New-Object -TypeName System.Net.WebClient).DownloadFile($url, $file)
powershell.exe -ExecutionPolicy Bypass -File $file -EnableCredSSP -ForceNewSSLC

# Verify WinRM listener
winrm enumerate winrm/config/Listener

# Verify WinRM service
Get-Service WinRM | Select-Object Status, StartType

# Open firewall for WinRM HTTPS

```

```
New-NetFirewallRule -Name "WinRM-HTTPS" -DisplayName "WinRM HTTPS" -Protocol TCP
```

Expected Output:

```
Listener
  Address = *
  Transport = HTTPS
  Port = 5986
  CertificateThumbprint = ABC123DEF456...
```

Inventory File

[File: /opt/ansible/inventory/production/hosts.yml]

```
---
all:
  children:
    windows_servers:
      children:
        windows_web:
          hosts:
            web-srv-01.domain.com:
            web-srv-02.domain.com:
            web-srv-03.domain.com:
        windows_app:
          hosts:
            app-srv-01.domain.com:
            app-srv-02.domain.com:
        windows_db:
          hosts:
            db-srv-01.domain.com:
            db-srv-02.domain.com:
```

[File: /opt/ansible/inventory/production/group_vars/windows_servers.yml]

```
---
ansible_user: svc_ansible@DOMAIN.COM
ansible_password: "{{ vault_ansible_password }}"
ansible_connection: winrm
ansible_winrm_transport: kerberos
ansible_winrm_server_cert_validation: ignore
ansible_port: 5986
ansible_winrm_scheme: https
```

Playbook: Install All Updates

[File: /opt/ansible/playbooks/patch_all_updates.yml]

```
---
- name: Install All Windows Updates
  hosts: windows_servers
  serial: 10
  gather_facts: yes
  tasks:
    - name: Install all available updates
```

```

ansible.windows.win_updates:
  category_names:
    - SecurityUpdates
    - CriticalUpdates
    - UpdateRollups
    - Updates
  state: installed
  reboot: yes
  reboot_timeout: 1800
  register: update_result

- name: Display update results
  ansible.builtin.debug:
    msg: "Installed {{ update_result.installed_update_count }} updates. Reboot required: {{ update_result.reboot_required }}"

```

Playbook: Security Updates Only

[File: /opt/ansible/playbooks/patch_security_only.yml]

```

---
- name: Install Security Updates Only
  hosts: windows_servers
  serial: 5
  tasks:
    - name: Install security updates
      ansible.windows.win_updates:
        category_names:
          - SecurityUpdates
        state: installed
        reboot: no
        register: update_result

    - name: Reboot if required (with timeout)
      ansible.windows.win_reboot:
        reboot_timeout: 600
        post_reboot_delay: 60
        when: update_result.reboot_required

```

Playbook: Install Specific KB

[File: /opt/ansible/playbooks/patch_specific_kb.yml]

```

---
- name: Install Specific KB Article
  hosts: windows_servers
  vars:
    kb_number: "KB5028168"
  tasks:
    - name: "Install {{ kb_number }}"
      ansible.windows.win_updates:
        accept_list:
          - "{{ kb_number }}"
        state: installed
        reboot: yes
        reboot_timeout: 900

```

Playbook: Batch Patching 500+ Servers

```
[File: /opt/ansible/playbooks/patch_batch_500.yml]
```

```
---
- name: Batch Patch 500+ Windows Servers (Rolling)
  hosts: windows_servers
  serial: 25
  max_fail_percentage: 10
  any_errors_fatal: false
  tasks:
    - name: Pre-check - Verify disk space
      ansible.windows.win_shell: |
        $disk = Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'"
        if (($disk.FreeSpace / 1GB) -lt 5) { exit 1 }
      register: disk_check
      failed_when: disk_check.rc != 0

    - name: Install all security and critical updates
      ansible.windows.win_updates:
        category_names:
          - SecurityUpdates
          - CriticalUpdates
        state: installed
        reboot: yes
        reboot_timeout: 1800
      register: update_result

    - name: Post-check - Verify services
      ansible.windows.win_shell: |
        $services = @('W32Time','WinRM','Dnscache')
        foreach ($svc in $services) {
          $s = Get-Service $svc -ErrorAction SilentlyContinue
          if ($s.Status -ne 'Running') { exit 1 }
        }

    - name: Log results
      ansible.builtin.lineinfile:
        path: /opt/ansible/logs/patch_results_{{ ansible_date_time.date }}.log
        line: "{{ inventory_hostname }} - Installed: {{ update_result.installed }}"
        create: yes
      delegate_to: localhost
```

Running Ansible Playbooks

[Run on: Ansible Control Node (Linux)]

```
# Dry run (check mode)
ansible-playbook /opt/ansible/playbooks/patch_security_only.yml --check -i /opt/ansible/inventory

# Execute patching
ansible-playbook /opt/ansible/playbooks/patch_security_only.yml -i /opt/ansible/inventory

# Limit to specific servers
ansible-playbook /opt/ansible/playbooks/patch_all_updates.yml -i /opt/ansible/inventory

# Verbose output for troubleshooting
ansible-playbook /opt/ansible/playbooks/patch_all_updates.yml -i /opt/ansible/inventory -v
```

Section 7: PDQ Deploy (Alternative for SMB)

What is PDQ Deploy?

PDQ Deploy is a software deployment tool designed for small-to-medium businesses. It pushes updates to Windows machines over the network without requiring agents on every endpoint.

When to Use

- Small environments (1-500 computers)
- No Active Directory required (workgroup support)
- Need quick, GUI-based deployment
- Budget-friendly alternative to SCCM

Installation

[Run on: PDQ Deploy Server (Windows)]

1. Download from: <https://www.pdq.com/pdq-deploy/>
2. Run installer: PDQDeploySetup.exe
3. Install path: C:\Program Files\Admin Arsenal\PDQ Deploy
4. License: Enter Enterprise key (or use Free mode)
5. Database location: C:\ProgramData\Admin Arsenal\PDQ Deploy\Database.db

Creating Patch Deployment Package

1. Open PDQ Deploy Console
2. Click: New Package
3. Steps → Add Step → Install → Windows Update
4. Configure:
 - Categories: Security, Critical
 - Reboot: After all steps if needed
5. Save as: "Monthly Security Patches"

Scheduling

1. Select package → Deploy Once or Schedule
2. Targets: Add from AD, CSV, or manual entry
3. Schedule: 3rd Saturday at 02:00
4. Options:
 - Offline retry: 4 attempts
 - Timeout: 2 hours per target
 - Bandwidth throttle: 50% (optional)

Reporting

- PDQ Inventory Integration:
1. Open PDQ Inventory → Reports
 2. Built-in report: "Windows Updates - Missing"
 3. Custom report: Filter by KB number, install date
 4. Export: PDF, CSV, HTML

Section 8: Pre-Patching Checklist

Automated Pre-Check Commands

[Run on: Target Windows Server]

Check Disk Space (Minimum 5 GB free on C:)

```
# Check free disk space
Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'" |
    Select-Object DeviceID,
    @{N='FreeGB';E={ [math]::Round($_.FreeSpace/1GB,2) }},
    @{N='TotalGB';E={ [math]::Round($_.Size/1GB,2) }},
    @{N='PercentFree';E={ [math]::Round((($_.FreeSpace/$_.Size)*100,2) }}
```

Expected Output:

```
DeviceID FreeGB TotalGB PercentFree
-----
C:         45.23  100.00  45.23
```

Check Critical Services

```
# Verify critical services are running
$services = @('wuauserv','CryptSvc','BITS','msiserver','WinRM','W32Time')
foreach ($svc in $services) {
    Get-Service -Name $svc | Select-Object Name, Status, StartType
}
```

Check Pending Reboots

```
# Check if reboot is pending
$pendingReboot = @()
if (Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUp
    $pendingReboot += "Windows Update"
}
if (Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Component
    $pendingReboot += "CBS"
}
if ($pendingReboot.Count -gt 0) {
    Write-Warning "REBOOT PENDING: $($pendingReboot -join ', ')"
} else {
    Write-Host "No pending reboot - OK to proceed" -ForegroundColor Green
}
```

Check Current Patch Level

```
# Get last installed updates
Get-HotFix | Sort-Object InstalledOn -Descending | Select-Object -First 5 HotFi
```

```
# Get OS build number
[System.Environment]::OSVersion.Version
(Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion").Display
```

Backup Verification

```
# Verify last backup status (Windows Server Backup)
Get-WBSummary | Select-Object LastSuccessfulBackupTime, LastBackupResultHR

# For System State backup
wbadmin get status
```

Create VM Snapshot (if applicable)

[Run on: VMware vCenter / Hyper-V Host]

```
# VMware (PowerCLI)
New-Snapshot -VM "ServerName" -Name "Pre-Patch-$(Get-Date -Format 'yyyyMMdd') "

# Hyper-V
Checkpoint-VM -Name "ServerName" -SnapshotName "Pre-Patch-$(Get-Date -Format 'y
```

Application Health Check

```
# Check IIS (if web server)
Get-Website | Select-Object Name, State, PhysicalPath

# Check SQL Server (if DB server)
Get-Service -Name "MSSQLSERVER" | Select-Object Status

# Test connectivity to critical endpoints
Test-NetConnection -ComputerName "app-server.domain.com" -Port 443
```

Section 9: Post-Patching Validation

[Run on: Target Windows Server]

Verify Installed Updates

```
# Show last 10 installed updates
Get-HotFix | Sort-Object InstalledOn -Descending | Select-Object -First 10 HotF

# Verify specific KB was installed
Get-HotFix -Id "KB5028168" -ErrorAction SilentlyContinue
if ($?) { Write-Host "KB5028168 installed successfully" -ForegroundColor Green
else { Write-Warning "KB5028168 NOT found!" }
```

Verify Services Are Running

```
# Check critical services post-reboot
$criticalServices = @('wuauserv','CryptSvc','BITS','W32Time','WinRM','Dnscache')
$failed = @()
foreach ($svc in $criticalServices) {
    $status = (Get-Service -Name $svc -ErrorAction SilentlyContinue).Status
    if ($status -ne 'Running') { $failed += "$svc ($status)" }
}
if ($failed.Count -eq 0) { Write-Host "All services OK" -ForegroundColor Green }
else { Write-Warning "Failed services: $($failed -join ', ')" }
```

Check Event Log for Errors

```
# Check System log for errors in last 2 hours
Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=(Get-Date).AddHours(-2)}
Select-Object TimeCreated, Id, Message | Format-Table -Wrap

# Check Application log
Get-WinEvent -FilterHashtable @{LogName='Application'; Level=2; StartTime=(Get-Date).AddHours(-2)}
Select-Object TimeCreated, Id, Message
```

Verify Reboot Completed

```
# Check last boot time
$lastBoot = (Get-CimInstance Win32_OperatingSystem).LastBootUpTime
Write-Host "Last Boot: $lastBoot"
Write-Host "Uptime: $((Get-Date) - $lastBoot)"

# Confirm no pending reboot remains
$pending = Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\PendingReboot"
Write-Host "Pending Reboot: $pending"
```

Application Health Verification

```
# Test web application (IIS)
$response = Invoke-WebRequest -Uri "http://localhost" -UseBasicParsing -Timeout 30
Write-Host "Web Server Status: $($response.StatusCode)"

# Test SQL connectivity
$conn = New-Object System.Data.SqlClient.SqlConnection "Server=localhost;Database=master;"
$conn.Open(); Write-Host "SQL: Connected"; $conn.Close()

# Test network connectivity
Test-NetConnection -ComputerName "dc01.domain.com" -Port 389
Test-NetConnection -ComputerName "app01.domain.com" -Port 443
```

Section 10: Rollback Procedures

Method 1: Uninstall Specific KB

[Run on: Target Windows Server]

```
# Uninstall using WUSA (recommended)
wusa /uninstall /kb:5028168 /quiet /norestart

# Uninstall using DISM
DISM /Online /Remove-Package /PackageName:Package_for_KB5028168~31bf3856ad364e3

# List installed packages to find exact name
DISM /Online /Get-Packages | findstr "KB5028168"
```

Method 2: DISM Rollback

```
# List all installed updates with DISM
DISM /Online /Get-Packages /Format:Table | Where-Object { $_ -match "Install" }

# Remove specific package
DISM /Online /Remove-Package /PackageName:"Package_for_KB5028168~31bf3856ad364e3

# Restore health if rollback causes issues
DISM /Online /Cleanup-Image /RestoreHealth
```

Method 3: Snapshot Revert

[Run on: VMware vCenter]

```
# VMware PowerCLI - Revert to pre-patch snapshot
Get-VM "ServerName" | Get-Snapshot -Name "Pre-Patch-*" | Sort-Object Created -D
```

[Run on: Hyper-V Host]

```
# Hyper-V - Restore checkpoint
Restore-VMSnapshot -VMName "ServerName" -Name "Pre-Patch-20260620" -Confirm:$fa
Start-VM -Name "ServerName"
```

Method 4: System Restore

[Run on: Target Windows Server]

```
# List available restore points
Get-ComputerRestorePoint | Select-Object SequenceNumber, Description, CreationT

# Restore to specific point (requires restart)
Restore-Computer -RestorePoint 15 -Confirm
```

⚠ Warning: System Restore is generally disabled on servers. Use VM snapshots or WUSA uninstall instead.

Section 11: Troubleshooting

30+ Common Patching Issues and Resolutions

#	Issue	Cause	Resolution
1	Windows Update stuck at 0%	Corrupted WU cache	Stop wuauserv, delete C:\Windows\SoftwareDistribution\Download*, restart
2	Windows Update stuck at downloading	BITS service issue	Restart-Service BITS -Force
3	Error 0x80070002	Missing files	DISM /Online /Cleanup-Image /RestoreHealth
4	Error 0x800f0831	Missing delta base	Install latest SSU, then retry CU
5	Error 0x80244010	WSUS max clients exceeded	Increase IIS connection limits on WSUS
6	Error 0x8024401c	WSUS unreachable	Verify http://wsus-server:8530 accessible
7	SCCM client not reporting	Client corrupted	Repair: ccmsetup.exe /remediate:client
8	SCCM updates showing 0% compliance	Policy not received	Run machine policy cycle, check PolicyAgent.log
9	WSUS sync failed	Connection timeout	Check proxy, verify wsusutil.exe reset
10	WSUS DB full	WID size limit (10 GB)	Run WSUS Cleanup Wizard, decline superseded
11	WinRM connection refused	Service not running	Enable-PSRemoting -Force on target
12	WinRM certificate error	Self-signed cert expired	Regenerate: winrm invoke Create winrm/config/Listener
13	WinRM Kerberos auth failed	SPN issue	setspn -L hostname, verify DNS
14	Patch KB fails to install	Pre-requisite missing	Check CBS log: C:\Windows\Logs\CBS\CBS.log
15	CBS corruption detected	Component store damaged	DISM /Online /Cleanup-Image /RestoreHealth /Source:wim:D:\sources\install.wim:1
16	Reboot loop after patch	Driver conflict	Boot Safe Mode, uninstall last KB

17	Blue screen (BSOD) after patch	Incompatible update	Boot recovery, DISM /Image:C:\ /Remove Package /PackageName:...
18	Insufficient disk space	C: drive full	Clean: Dism.exe /Online /Cleanup-Image /StartComponentCleanup /ResetBase
19	.NET update failed	Old .NET version conflict	Uninstall old .NET, install latest
20	Feature update stuck at restart	Pending operations	Clear C:\Windows\WinSxS\pending.xml
21	SCCM DP content hash mismatch	Corrupted package	Redistribute package to DP
22	Updates install but revert on reboot	Trusted installer issue	Check C:\Windows\Logs\CBS\CBS.log for "Failure"
23	Ansible win_updates timeout	Update takes too long	Increase reboot_timeout: 3600
24	Ansible "unreachable" host	WinRM/firewall issue	Test: Test-NetConnection target -Port 5986
25	WSUS clients in wrong group	GPO targeting issue	wuautil /resetauthorization /detectnow
26	Update installs but not detected	WMI corruption	Rebuild WMI: winmgmt /resetrepository
27	SCCM maintenance window missed	Time zone mismatch	Verify MW uses UTC vs local time
28	Multiple reboots required	Stacked updates	Normal behavior; install SSU first, then CU
29	Windows Update service won't start	Dependency broken	Check RPCSS and CryptSvc dependencies
30	Intune update ring not applying	Device not enrolled	Verify: dsregcmd /status
31	Patch causes application crash	Compatibility issue	Test in staging first, rollback KB

32	SCCM client cache full	Old cached content	Clear cache: SCCM Control Panel → Cache → Delete

Detailed Resolution Commands

Fix Windows Update Stuck

[Run on: Target Windows Server]

```
# Stop services
Stop-Service wuauserv, cryptSvc, bits, msiserver -Force

# Clear update cache
Remove-Item "C:\Windows\SoftwareDistribution\*" -Recurse -Force
Remove-Item "C:\Windows\System32\catroot2\*" -Recurse -Force

# Re-register DLLs
regsvr32.exe /s atl.dll
regsvr32.exe /s urlmon.dll
regsvr32.exe /s mshtml.dll
regsvr32.exe /s shdocvw.dll
regsvr32.exe /s wintrust.dll
regsvr32.exe /s wuapi.dll
regsvr32.exe /s wuaueng.dll
regsvr32.exe /s wups.dll

# Restart services
Start-Service wuauserv, cryptSvc, bits, msiserver
```

Fix CBS Corruption

[Run on: Target Windows Server]

```
# Step 1: Run SFC
sfc /scannow

# Step 2: If SFC fails, run DISM
DISM /Online /Cleanup-Image /CheckHealth
DISM /Online /Cleanup-Image /ScanHealth
DISM /Online /Cleanup-Image /RestoreHealth

# Step 3: If DISM fails, use Windows media as source
DISM /Online /Cleanup-Image /RestoreHealth /Source:wim:E:\sources\install.wim:1

# Step 4: Run SFC again
sfc /scannow
```

Log locations: - SFC: C:\Windows\Logs\CBS\CBS.log - DISM: C:\Windows\Logs\DISM\dism.log

Fix SCCM Client Not Reporting

[Run on: Target Windows Server]

```
# Check client health
Get-Service CcmExec | Select-Object Status

# Repair client
& "C:\Windows\ccmsetup\ccmsetup.exe" /remediate:client

# Full reinstall if repair fails
& "C:\Windows\ccmsetup\ccmsetup.exe" /uninstall
Start-Sleep -Seconds 300
& "\\sccm-server\Client\ccmsetup.exe" SMSITECODE=PS1 SMSMP=sccm-server.domain.
```

Fix WSUS Sync Issues

[Run on: WSUS Server]

```
# Reset WSUS
& "C:\Program Files\Update Services\Tools\wsusutil.exe" reset

# Clean up WSUS database
Invoke-WsusServerCleanup -CleanupObsoleteUpdates -CleanupUnneededContentFiles

# Check IIS Application Pool
Get-WebAppPoolState -Name "WsusPool"
Start-WebAppPool -Name "WsusPool"
```

Section 12: Automation Scripts

PowerShell: Complete Pre-Patch Check Script

[Run on: Target Windows Server] [Save as: C:\Scripts\Pre-Patch-Check.ps1]

```
#Requires -RunAsAdministrator
<#
.SYNOPSIS
    Pre-patching health check script for Windows Server
.DESCRIPTION
    Validates server readiness before applying Windows updates
#>

$results = @()
$serverName = $env:COMPUTERNAME
$timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

Write-Host "=" * 60
Write-Host " PRE-PATCH CHECK - $serverName - $timestamp" -ForegroundColor Cyan
Write-Host "=" * 60

# 1. Disk Space Check
$disk = Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'"
$freeGB = [math]::Round($disk.FreeSpace / 1GB, 2)
$status = if ($freeGB -ge 5) { "PASS" } else { "FAIL" }
Write-Host "[Disk Space] C: Free: ${freeGB} GB - $status" -ForegroundColor $(if
$results += [PSCustomObject]@{Check="Disk Space"; Result=$status; Detail="${fre

# 2. Pending Reboot
```



```

$pendingReboot = $false
if (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\AutoDownload\Branch\changelog.txt") {
    if (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\RebootPending") {
        $status = if (-not $pendingReboot) { "PASS" } else { "WARNING" }
        Write-Host "[Pending Reboot] $pendingReboot - $status" -ForegroundColor $(if($status -eq "PASS") { "Green" } else { "Yellow" })
        $results += [PSCustomObject]@{Check="Pending Reboot"; Result=$status; Detail="Pending Reboot"}
    }
}

# 3. Critical Services
$criticalSvcs = @('wuauserv','CryptSvc','BITS','WinRM','W32Time')
$svcFailed = @()
foreach ($svc in $criticalSvcs) {
    $s = Get-Service -Name $svc -ErrorAction SilentlyContinue
    if ($s.Status -ne 'Running') { $svcFailed += $svc }
}
$status = if ($svcFailed.Count -eq 0) { "PASS" } else { "FAIL" }
Write-Host "[Services] Failed: $($svcFailed -join ', ') - $status" -ForegroundColor $(if($status -eq "PASS") { "Green" } else { "Red" })
$results += [PSCustomObject]@{Check="Critical Services"; Result=$status; Detail="$svcFailed"}

# 4. Last Patch Date
$lastPatch = (Get-HotFix | Sort-Object InstalledOn -Descending | Select-Object -First 1).InstalledOn
$daysSince = ((Get-Date) - $lastPatch).Days
Write-Host "[Last Patch] $lastPatch ($daysSince days ago)" -ForegroundColor Cyan
$results += [PSCustomObject]@{Check="Last Patch"; Result="INFO"; Detail="$lastPatch"}

# 5. OS Build
$build = (Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion").CurrentBuild
Write-Host "[OS Build] $($build.ProductName) - Build $($build.CurrentBuildNumber)" -ForegroundColor Cyan
$results += [PSCustomObject]@{Check="OS Build"; Result="INFO"; Detail="$build"}

# Summary
Write-Host "`n" + "=" * 60
$results | Format-Table -AutoSize
if ($results | Where-Object Result -eq "FAIL") {
    Write-Host "❌ PRE-CHECK FAILED - DO NOT PROCEED WITH PATCHING" -ForegroundColor Red
    exit 1
} else {
    Write-Host "✅ PRE-CHECK PASSED - OK TO PROCEED" -ForegroundColor Green
    exit 0
}

```

PowerShell: Post-Patch Validation Script

[Run on: Target Windows Server] [Save as: C:\Scripts\Post-Patch-Validate.ps1]

```

#Requires -RunAsAdministrator
<#
.SYNOPSIS
    Post-patching validation script
#>

$serverName = $env:COMPUTERNAME
Write-Host "=" * 60
Write-Host " POST-PATCH VALIDATION - $serverName" -ForegroundColor Cyan
Write-Host "=" * 60

# 1. Verify new patches installed
Write-Host "`n[Recent Updates]" -ForegroundColor Yellow
Get-HotFix | Sort-Object InstalledOn -Descending | Select-Object -First 10 HotFixID, InstalledOn, Description

```

```
# 2. Check uptime (should be < 1 hour if just rebooted)
$lastBoot = (Get-CimInstance Win32_OperatingSystem).LastBootUpTime
$uptime = (Get-Date) - $lastBoot
Write-Host "[Uptime] Last Boot: $lastBoot (Uptime: $($uptime.Hours)h $($uptime.Minutes)m $($uptime.Seconds)s)"

# 3. Services check
$criticalSvcs = @('wuauserv','CryptSvc','BITS','WinRM','W32Time','Dnscache','La
$svcFailed = @()
foreach ($svc in $criticalSvcs) {
    $s = Get-Service -Name $svc -ErrorAction SilentlyContinue
    if ($s -and $s.Status -ne 'Running') { $svcFailed += "$svc($($s.Status))" }
}
if ($svcFailed.Count -eq 0) { Write-Host "[Services] All critical services running" }
else { Write-Host "[Services] FAILED: $($svcFailed -join ', ') ✖" -ForegroundColor Red }

# 4. Event log errors (last 1 hour)
$errors = Get-WinEvent -FilterHashtable @{LogName='System'; Level=2; StartTime=$(Get-Date).AddHours(-1)}
if ($errors) {
    Write-Host "[Event Log] $($errors.Count) errors in last hour:" -ForegroundColor Red
    $errors | Select-Object TimeCreated, Id, Message | Format-Table -Wrap
} else {
    Write-Host "[Event Log] No critical errors ✔" -ForegroundColor Green
}

# 5. Pending reboot check
$pending = Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\
if ($pending) { Write-Host "[Reboot] Still pending - additional reboot required" }
else { Write-Host "[Reboot] No pending reboot ✔" -ForegroundColor Green }

Write-Host "`n" + "=" * 60
Write-Host " VALIDATION COMPLETE" -ForegroundColor Cyan
```

Ansible: Full Patching Workflow Playbook

[File: /opt/ansible/playbooks/full_patch_workflow.yml]

```
---
- name: Complete Windows Patching Workflow
  hosts: windows_servers
  serial: 10
  vars:
    patch_categories:
      - SecurityUpdates
      - CriticalUpdates
    log_path: /opt/ansible/logs
  tasks:
    # PRE-PATCH CHECKS
    - name: "PRE-CHECK: Verify disk space (min 5 GB)"
      ansible.windows.win_shell: |
        $free = [math]::Round((Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'").FreeSpace / 1GB)
        if ($free -lt 5) { Write-Error "Only ${free}GB free"; exit 1 }
        Write-Output "Disk OK: ${free}GB free"
      register: disk_check

    - name: "PRE-CHECK: Verify no pending reboot"
      ansible.windows.win_shell: |
        $pending = Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\
        if ($pending) { Write-Error "Reboot pending"; exit 1 }
        Write-Output "No pending reboot"
      register: reboot_check
```

```

- name: "PRE-CHECK: Create VM snapshot"
  ansible.windows.win_shell: |
    Write-Output "Snapshot requested - verify with VMware admin"
  register: snapshot_check

# PATCHING
- name: "PATCH: Install updates"
  ansible.windows.win_updates:
    category_names: "{{ patch_categories }}"
    state: installed
    reboot: yes
    reboot_timeout: 1800
  register: patch_result

# POST-PATCH VALIDATION
- name: "POST-CHECK: Verify services running"
  ansible.windows.win_shell: |
    $failed = @()
    @('wuauserv','CryptSvc','BITS','WinRM') | ForEach-Object {
      if ((Get-Service $_).Status -ne 'Running') { $failed += $_ }
    }
    if ($failed) { Write-Error "Failed: $($failed -join ', ')" ; exit 1 }
    Write-Output "All services OK"
  register: svc_check

- name: "POST-CHECK: Check for errors in event log"
  ansible.windows.win_shell: |
    $errs = Get-WinEvent -FilterHashtable @{LogName='System';Level=2;StartT
    if ($errs) { Write-Warning "$($errs.Count) errors found" }
    else { Write-Output "No errors" }
  register: eventlog_check

# REPORTING
- name: "REPORT: Log results"
  ansible.builtin.lineinfile:
    path: "{{ log_path }}/patch_report_{{ ansible_date_time.date }}.csv"
    line: "{{ inventory_hostname }},{{ patch_result.installed_update_count"
    create: yes
    delegate_to: localhost

```

Section 13: Best Practices & Common Mistakes

✓ Best Practices

#	Practice	Detail
1	Test in staging first	Always deploy to non-production 7 days before production
2	Use maintenance windows	Patch during off-hours (Saturday 02:00-06:00 recommended)
3	Take snapshots	VM snapshots before patching; delete within 72 hours
4	Patch in waves	Wave 1: Dev → Wave 2: Staging → Wave 3: Production
5	Monitor after patching	Watch for 24-48 hours post-patch for issues

6	Keep SSU current	Always install Servicing Stack Updates before Cumulative
7	Document everything	Record KB numbers, servers patched, issues encountered
8	Automate pre/post checks	Use scripts from Section 12 every time
9	Maintain a rollback plan	Know exactly how to undo before you start
10	Patch monthly	Never skip more than 2 months; cumulative debt grows
11	Exclude critical apps	Separate database and domain controller patching
12	Communicate	Send pre-patch and post-patch notifications

✗ Common Mistakes

#	Mistake	Impact	Correction
1	Patching without snapshot	No rollback path	Always snapshot VMs first
2	Patching all servers at once	Total outage risk	Use serial/wave approach
3	Ignoring pending reboots	Patch install failure	Reboot first, then patch
4	Not testing patches	Application breakage	7-day staging test period
5	Skipping pre-checks	Patch fails mid-install	Run pre-check script always
6	Force-rebooting during install	OS corruption	Wait for completion
7	Not monitoring post-patch	Missed silent failures	Check logs for 24 hours
8	Deleting SoftwareDistribution randomly	Breaks WU metadata	Only delete Download subfolder
9	Using Feature Updates on servers	Unexpected major upgrade	Only apply CUs to servers
10	Not backing up before DC patching	AD corruption risk	Full system state backup

Section 14: Change Management

ITIL-Aligned Patch Change Process



Change Request Template

Field	Value
Change ID	CHG-2026-0623-001
Title	Monthly Windows Security Patching - June 2026
Type	Standard (Pre-approved for monthly patching)
Category	Infrastructure → Windows → Patching
Priority	Medium
Risk	Low (tested in staging)
Environment	Production Windows Servers
Servers Affected	150 (list attached)
Scheduled Start	Saturday 2026-06-27 02:00 UTC
Scheduled End	Saturday 2026-06-27 06:00 UTC
Rollback Plan	Revert VM snapshot or uninstall KB via WUSA
Backout Time	30 minutes per server
Testing Evidence	Staging patched 2026-06-20, no issues after 7 days
Approver	Change Advisory Board
Implementer	Windows Platform Team

Patching Timeline

Day	Activity
Patch Tuesday (Day 0)	Microsoft releases patches
Day 1-2	Review patches, identify critical KBs
Day 3-7	Deploy to Dev/Test environment
Day 7-14	Monitor staging, run validation
Day 14	Submit Change Request for Production
Day 15-16	CAB review and approval
Day 17-21	Production Wave 1 (non-critical servers)
Day 21-28	Production Wave 2 (critical servers)
Day 28-30	Post-Implementation Review (PIR)

Section 15: Quick Reference / Cheat Sheet

Essential Commands at a Glance

Command	Where to Run	Purpose
<code>Get-HotFix \ Sort-Object InstalledOn -Descending \ Select -First 10</code>	Target Server	View recent patches
<code>wusa /uninstall /kb:XXXXXXX /quiet /norestart</code>	Target Server	Uninstall specific KB
<code>usoclient StartScan</code>	Target Server	Force update scan (2019+)
<code>wuaclt /detectnow /reportnow</code>	Target Server	Force WSUS detection (legacy)
<code>DISM /Online /Cleanup-Image /RestoreHealth</code>	Target Server	Fix component store
<code>sfc /scannow</code>	Target Server	Fix system files
<code>Restart-Service wuauserv -Force</code>	Target Server	Restart Windows Update
<code>Get-Service CcmExec</code>	Target Server	Check SCCM client
<code>Invoke-WsusServerCleanup</code>	WSUS Server	Clean WSUS database
<code>wsusutil.exe reset</code>	WSUS Server	Reset WSUS sync
<code>ansible-playbook patch.yml -i hosts.yml</code>	Ansible Control Node	Run patching playbook
<code>ansible-playbook patch.yml --check</code>	Ansible Control Node	Dry run
<code>winrm enumerate winrm/config/Listener</code>	Target Server	Verify WinRM config
<code>Test-NetConnection host -Port 5986</code>	Any Server	Test WinRM connectivity
<code>Get-WindowsUpdateLog</code>	Target Server	Generate readable WU log
<code>Get-WinEvent -LogName System -MaxEvents 20</code>	Target Server	Check system events

Critical File Paths

Path	Description
<code>C:\Windows\SoftwareDistribution</code>	Windows Update download cache

C:\Windows\SoftwareDistribution\Download	Downloaded update files
C:\Windows\System32\catroot2	Cryptographic catalog cache
C:\Windows\Logs\CBS\CBS.log	Component Based Servicing log
C:\Windows\Logs\DISM\dism.log	DISM operation log
C:\Windows\WindowsUpdate.log	Windows Update log (legacy)
C:\Windows\CCM\Logs\	SCCM client logs directory
C:\Windows\CCM\Logs\UpdatesHandler.log	SCCM update handler log
C:\Windows\CCM\Logs\WUAHandler.log	SCCM WUA interaction log
D:\WSUS_Content	WSUS content storage
C:\Program Files\Update Services\LogFiles	WSUS server logs
/etc/ansible/ansible.cfg	Ansible global config
/opt/ansible/playbooks/	Ansible playbooks directory
/opt/ansible/inventory/	Ansible inventory directory
\\sccm-server\Sources\Updates\YYYY-MM	SCCM update package source

Service Dependencies

Service	Display Name	Must Be Running
wuauserv	Windows Update	✓ Yes
BITS	Background Intelligent Transfer	✓ Yes
CryptSvc	Cryptographic Services	✓ Yes
msiserver	Windows Installer	✓ During install
TrustedInstaller	Windows Modules Installer	✓ During install
CcmExec	SMS Agent Host (SCCM)	✓ If using SCCM
WinRM	Windows Remote Management	✓ If using Ansible

⚠ Disclaimer

This document is provided for informational and educational purposes only. Always test patches in a non-production environment before deploying to production. The authors are not responsible for any data loss, downtime, or damage resulting from the use of procedures described in this guide.

- Verify all commands against your specific environment before

execution.

- File paths may vary based on installation choices and OS versions.
- Always follow your organization's change management policies.
- Keep this document updated as Microsoft releases new tools and changes processes.

Last Updated: June 2026 | **Version:** 3.0 | **Author:** Windows Platform Engineering Team

End of Windows Patch Management Guide